

Esercizi sulla Tecnica Divide et Impera.

Ugo Vaccaro

1. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, non necessariamente ordinato. Si scriva un algoritmo basato sulla tecnica Divide et Impera che conti il numero di elementi in A che sono ≥ 0 . Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

2. Si progetti un algoritmo basato sulla tecnica Divide et Impera, e se ne analizzi la complessità, per il seguente problema algoritmico:

Input: Array $A = A[1..n]$ di n numeri interi positivi.

Output:

$$\min_{1 \leq i \leq j} (A[j] - A[i]).$$

(Suggerimento: si proceda come fatto a lezione per il calcolo di $\max_{1 \leq i \leq j} (A[j] - A[i])$.)

◇

3. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, *non* necessariamente distinti e non necessariamente ordinati. Si scriva un algoritmo basato sulla tecnica Divide et Impera che, dato A e due numeri qualsiasi $L < U$, calcoli quanti valori di A appartengono all'intervallo $[L, U]$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

4. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, distinti tra di loro ed ordinati in senso crescente. Si scriva un algoritmo basato sulla tecnica Divide et Impera che, dato A e un numero arbitrario x , restituisce l'indice del più piccolo elemento in A che risulti strettamente maggiore di x . Se non esiste alcun valore di A che soddisfi tale vincolo, l'algoritmo deve restituire “non c'è”. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

5. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, distinti tra di loro ed ordinati in senso crescente. Si scriva un algoritmo basato sulla tecnica Divide et Impera che, dato A e due numeri qualsiasi $L < U$, calcoli quanti valori di A appartengono all'intervallo $[L, U]$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

6. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, ordinati in senso non decrescente. Si scriva un algoritmo basato sulla tecnica Divide et Impera che per decidere se A contiene oppure no elementi duplicati. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

7. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di numeri, non necessariamente ordinato. Si scriva un algoritmo basato sulla tecnica Divide et Impera che conti il numero di indici i per cui valga $A[i] \times A[i+1] > 0$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

8. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore ordinato in senso crescente che è stato shiftato k posizioni a sinistra. Ad esempio, il vettore $[15, 18, 28, 30, 35, 42, 1, 7]$ è un vettore ordinato che è stato shiftato $k = 2$ posizioni a sinistra, mentre il vettore $[30, 35, 42, 1, 7, 15, 18, 28]$ è un vettore ordinato che è stato shiftato $k = 5$ posizioni a sinistra.

- (a) Supponendo di avere A e k in input, dare un algoritmo che determina il minimo in A in tempo $O(1)$
- (b) Supponendo di avere *solo* il vettore A in input, dare un algoritmo che determina il minimo in A in tempo $O(\log n)$

◇

9. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore ordinato in senso crescente che è stato shiftato k posizioni a sinistra. Avendo in input il solo vettore A , progettare un algoritmo basato sulla tecnica Divide et Impera che determini il valore k di cui sopra. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

10. *Esercizio:* Sia A un vettore di n interi. Si supponga che *entrambe* le seguenti due condizioni valgano:

- per ogni $i = 1, 2, \dots, n-1$, si ha che $|A[i+1] - A[i]| \leq 1$,
- $A[1] \leq 0$ e $A[n] > 0$.

Si dice zero del vettore un indice k tale che $A[k] = 0$. Dato un siffatto vettore A di $n \geq 2$ interi, provare che A ha almeno uno zero. Progettare ed analizzare un algoritmo basato sulla tecnica Divide et Impera che trovi uno zero di A in tempo $O(\log n)$.

◇

11. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di n interi, con $0 > A[1] < A[2] < \dots < A[n] > 0$. Progettare un algoritmo basato sulla tecnica Divide et Impera che calcoli il più piccolo valore di i per cui $A[i] < 0$ e $A[i+1] > 0$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

12. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di n interi, che possono essere sia positivi che negativi. Si supponga che $A[1] \leq \dots \leq A[n]$. Progettare un algoritmo basato sulla tecnica Divide et Impera che conti il numero di elementi > 0 nel vettore A . Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

13. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di n interi. Progettare un algoritmo basato sulla tecnica Divide et Impera che conti il numero di elementi $A[i]$ nel vettore A , $1 < i < n$, per cui valga sia che $A[i] = A[i+1]$ che $A[i] = A[i-1]$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

14. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore ordinato in senso crescente tale che $A[i] \in \{0, 1\}$, per ogni $i = 1, \dots, n$. Progettare un algoritmo basato sulla tecnica Divide et Impera che conti il numero di elementi $A[i]$ nel vettore A , per cui valga che $A[i] = 1$. Se ne analizzi la complessità. (Il massimo del punteggio lo si ottiene per un algoritmo di complessità $O(\log n)$).

◇

15. *Esercizio:* Siano A ed n numeri naturali positivi. Progettare un algoritmo basato sulla tecnica Divide et Impera che calcoli nA , utilizzando $O(\log n)$ addizioni.

◇

16. *Esercizio:* Sia $A = A[1] \dots A[n]$ un vettore di n interi, tale che $A[1] < \dots < A[n]$. L'indice $i \in \{2, \dots, n\}$ è detto un *salto*, se vale che $A[i-1] + 1 < A[i]$. Progettare un algoritmo che determini se il generico vettore A ha un salto, e nell'ipotesi che lo abbia, determini attraverso la tecnica Divide et Impera almeno un valore di i per cui $A[i-1] + 1 < A[i]$. Si analizzi la complessità di tempo dell'algoritmo proposto, giustificando le affermazioni fatte.

◇

17. *Esercizio:* Sia data una matrice $n \times n$ di interi in cui gli elementi di ogni riga sono ordinati in senso decrescente, e anche gli elementi di ogni colonna sono ordinati in senso decrescente. Si progetti un algoritmo per determinare se un dato intero k è presente nella matrice o meno, e se ne analizzi la complessità.

◇

18. *Esercizio:* Dato un vettore *ordinato* di interi $A[1 \dots n]$ ed un intero N , si progetti e si analizzi un algoritmo basato sulla tecnica Divide et Impera che, preso in input il vettore A ed il numero N determini se esistono o meno due indici i e j per cui $A[i] \times A[j] = N$. L'algoritmo deve avere complessità $O(N \log N)$.

◇

19. *Esercizio:* Si consideri il seguente problema: la Ditta ACME è stata quotata in borsa, ed il valore delle sue azioni sono state tabulate per tutto l'anno. Sia A il valore di una azione di ACME al primo Gennaio, e sia B il corrispondente valore al 31 Dicembre.

- (a) Se $A > B$, argomentare che c'è stato un giorno dell'anno in cui l'azione di ACME è stata quotata ad un valore inferiore al giorno precedente;

- (b) formalizzando opportunamente il problema in termini algoritmici (cioè definendo con precisione chi sono gli input e gli output al problema), sia dia un algoritmo di complessità logaritmica nella taglia dell'input che, sotto l'ipotesi che $A > B$, determini un giorno dell'anno in cui l'azione di ACME è stata quotata ad un valore inferiore al giorno precedente.

◇

20. *Esercizio:* Si consideri il seguente problema:

Input: array $a=a[1] \dots a[n]$ contenente valori interi ordinati in senso non decrescente (possono essere presenti valori duplicati), intero x .

Output: l'indice (la posizione) dell'**dell'ultima** occorrenza di x in $a=a[1] \dots a[n]$, oppure 0 se il valore x non è presente in $a=a[1] \dots a[n]$.

Si progetti e si analizzi un algoritmo basato sulla tecnica Divide et Impera che risolve il problema.

◇

21. *Esercizio:* Consideriamo il seguente problema algoritmico:

Input: vettore $a=a[1]a[2] \dots a[n]$, ordinato in senso crescente, di n numeri tutti distinti tra di loro tranne che per una coppia, con $a[i] \in \{1, 2, \dots, n-1\}$, per ogni $i = 1, \dots, n$

Output: l'unico intero $k \in \{1, 2, \dots, n-1\}$ che compare due volte in a

Suggerimento: si proceda come fatto a lezione per l'elemento mancante.

◇

22. *Esercizio:* Si descriva l'algoritmo `QuickSelect (a, sinistra, k, destra)` visto a lezione (non è necessario descrivere in dettaglio l'algoritmo `Distribuzione(a, sinistra, pivot, destra)`, si derivi l'equazione di ricorrenza che descrive il tempo medio di esecuzione di `QuickSelect`.

◇

23. *Esercizio:* Consideriamo il seguente problema. Data una sequenza di numeri $a = a[0]a[1] \dots a[n-1]$, un minimo relativo di a è un elemento di a che è *minore o uguale* dei suoi adiacenti (o di un **solo** adiacente, se stiamo considerando $a[0]$ o $a[n-1]$).

Si derivi e si analizzi un algoritmo basato sulla tecnica Divide et Impera per il seguente problema algoritmico.

Input: array $a = a[0]a[1] \dots a[n-1]$

Output: un (qualsivoglia) minimo relativo di a .

◇

24. *Esercizio:* Si ricordi l'equazione di ricorrenza che descrive il tempo medio di esecuzione $T(n)$ di `QuickSelect` (non è necessario derivarla) e si provi che $T(n) = O(n)$.

◇

25. *Esercizio:* Data una arbitraria sequenza di numeri $a = a[1]a[2] \dots a[n]$, si provi che ogni algoritmo di ordinamento per $a = a[1]a[2] \dots a[n]$ (basato su confronti) deve eseguire necessariamente $\Omega(n \log n)$ confronti nel caso peggiore.

◇

26. *Esercizio:* Sia $a = a[1]a[2] \dots a[n]$ sequenza arbitraria di numeri, $n > 100$, di cui si sa che i primi 10 elementi e gli ultimi 20 elementi sono *correttamente* ordinati. Si provi che ogni algoritmo di ordinamento per $a = a[1]a[2] \dots a[n]$ (basato su confronti) deve eseguire necessariamente $\Omega(n \log n)$ confronti nel caso peggiore.

◇

27. *Esercizio:* Supponiamo che esista un “coprocessore speciale” in grado di effettuare il merge di due sequenze di due liste ordinate di m elementi utilizzando $\sqrt{m}$ passi, e sia `specialmerge()` la funziona da chiamare per utilizzare tale coprocessore. Descrivere un algoritmo di ordinamento basato su tale funzione ed analizzare il suo tempo di esecuzione nel caso peggiore.

◇

28. *Esercizio:* Progettare un algoritmo che, prese in input due sequenze A e B di caratteri trova il massimo intero k tale che B^k è una sottosequenza di A . La sequenza B^k è quella che si ottiene ripetendo k volte ogni caratteri di B , ad esempio se $B = aab$ allora $B^3 = aaaaaabbb$. Una sequenza B è una sottosequenza di una sequenza A se B si può ottenere da A eliminando eventualmente dei caratteri. L’ algoritmo deve avere complessità $O(n \log n)$ dove n è la lunghezza di A .